

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CSA1610 **COURSE CODE:** Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets

QUESTIONS: 05

Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

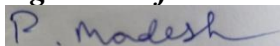
Criteria	Problem Understanding (2)	Algorithm Design (3)	Use of Control Structures (2)	Pseudocode Structure (1)	Completeness (2)	Total(10)
Weightage	20%	30%	20%	10%	20%	100%
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						
<i>Q4</i>						
<i>Q5</i>						

Code of Conduct:

I P. Madesh 192365007 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Q1. Dataset: Online Shopping Transactions

Order ID Products Purchased

O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question:

Design pseudocode for the **Apriori algorithm** to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration.

Q2. Dataset: Library Book Borrowing Records

Borrow ID Books Borrowed

B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question:

Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. (BL4)

Q3. Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3

Service Combination	Support Count
{Streaming, Cloud Storage}	2

Question:

Write pseudocode to generate **association rules** from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. (BL3)

Q4. Dataset: Student Course Enrollment

Student ID Courses Enrolled

S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question:

Design pseudocode for **constraint-based frequent itemset mining** where only itemsets containing the course "**Python**" are considered during the mining process. Explain how the constraint affects candidate generation and pruning. (BL4)

Q5. Dataset: Hospital Treatment Hierarchy

Patient ID Treatments Received

P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question:

Develop pseudocode for **multilevel association rule mining** using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

Question 1 — Apriori Algorithm (Online Shopping Transactions)

Dataset given:

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Assume a minimum support threshold of $\text{min_sup} = 3$ transactions (i.e. 60%, since $|D| = 5$).

1.1 Concept

Apriori is a level-wise, breadth-first algorithm that finds frequent itemsets using the anti-monotone (downward-closure) property: every subset of a frequent itemset must itself be frequent. It alternates between two phases at every iteration k : (a) generate candidate k -itemsets C_k by joining frequent $(k-1)$ -itemsets L_{k-1} with themselves, and (b) prune C_k using the downward-closure property and count support by scanning the database, keeping only itemsets that meet min_sup to form L_k .

1.2 Pseudocode

ALGORITHM Apriori($D, \text{min_sup}$)

INPUT : D // transaction database

min_sup // minimum support count

OUTPUT: L // all frequent itemsets

1. $C_1 \leftarrow \{\text{every single item in } D\}$
2. $L_1 \leftarrow \{c \text{ in } C_1 \mid \text{support_count}(c) \geq \text{min_sup}\}$ // scan D once
3. $k \leftarrow 2$
4. WHILE $L_{(k-1)}$ is not empty DO
5. $C_k \leftarrow \text{APRIORI-GEN}(L_{(k-1)})$ // candidate generation
6. FOR EACH transaction t in D DO
7. $C_t \leftarrow \text{SUBSET}(C_k, t)$ // candidates contained in t
8. FOR EACH candidate c in C_t DO
9. $c.\text{count} \leftarrow c.\text{count} + 1$
10. $L_k \leftarrow \{c \text{ in } C_k \mid c.\text{count} \geq \text{min_sup}\}$
11. $k \leftarrow k + 1$
12. RETURN $L \leftarrow \text{Union of all } L_k$

FUNCTION APRIORI-GEN($L_{(k-1)}$)

1. $C_k \leftarrow \text{empty set}$
2. FOR EACH pair p, q in $L_{(k-1)}$ DO
3. IF first $(k-2)$ items of p and q are identical
AND last item of $p <$ last item of q THEN
4. $\text{candidate} \leftarrow p \text{ JOIN } q$ // $(k-1)+(k-1) \rightarrow k$ items
5. // PRUNE STEP (downward closure)
6. IF every $(k-1)$ -subset of candidate is in $L_{(k-1)}$ THEN
7. $C_k \leftarrow C_k \cup \{\text{candidate}\}$

```

8.      ELSE
9.      discard candidate    // cannot be frequent
10. RETURN Ck

```

1.3 Worked trace on the given dataset ($\text{min_sup} = 3$)

Iteration 1 — C1 and L1

Itemset	Support Count	Frequent? (≥ 3)
{Mobile}	4 (O1,O2,O4,O5)	Yes
{Earphones}	4 (O1,O3,O4,O5)	Yes
{Charger}	3 (O2,O3,O4)	Yes

$L1 = \{ \{ \text{Mobile} \}, \{ \text{Earphones} \}, \{ \text{Charger} \} \}$ — all three survive since none is below min_sup .

Iteration 2 — Candidate generation C2 and pruning

APRIORI-GEN joins $L1$ with itself: $\{ \text{Mobile}, \text{Earphones} \}$, $\{ \text{Mobile}, \text{Charger} \}$, $\{ \text{Earphones}, \text{Charger} \}$. No 1-itemset subset can be missing from $L1$ ($k-2 = 0$), so no candidate is pruned at this step; all three move to the support-counting scan.

Candidate	Support Count	Frequent? (≥ 3)
{Mobile, Earphones}	3 (O1,O4,O5)	Yes
{Mobile, Charger}	2 (O2,O4)	No — pruned
{Earphones, Charger}	2 (O3,O4)	No — pruned

$L2 = \{ \{ \text{Mobile}, \text{Earphones} \} \}$ (support = 3).

Iteration 3 — Termination

APRIORI-GEN needs at least two 2-itemsets that share their first item to join into a 3-itemset. $L2$ has only one member, so $C3 = \text{empty set}$. The algorithm terminates.

Final answer:

- Frequent 1-itemsets: $\{ \text{Mobile} \}$, $\{ \text{Earphones} \}$, $\{ \text{Charger} \}$
- Frequent 2-itemset: $\{ \text{Mobile}, \text{Earphones} \}$
- No frequent 3-itemset exists at $\text{min_sup} = 3$.

This demonstrates the key efficiency gain of Apriori: $\{ \text{Mobile}, \text{Charger} \}$ and $\{ \text{Earphones}, \text{Charger} \}$ were counted but pruned by the support scan, and because $L2$ could not self-join, the database was never scanned for any 3-itemset — the downward-closure pruning avoided wasted work.

Question 2 — FP-Growth Algorithm (Library Book Borrowing Records)

Dataset given:

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Assume $\text{min_sup} = 3$ transactions (60%).

2.1 Concept

FP-Growth avoids the costly candidate-generate-and-test cycle of Apriori. It compresses the database into a highly compact prefix-tree, the FP-tree, built from only frequent items sorted in descending order of support. Frequent patterns are then mined directly from the tree using a divide-and-conquer strategy (pattern-growth via conditional pattern bases and conditional FP-trees), so candidate itemsets are never explicitly generated and the database is scanned only twice.

2.2 Pseudocode

(a) Building the FP-tree

```
ALGORITHM Build_FP_Tree(D, min_sup)
1. Scan D once; compute support_count of every item.
2. L <- {items with support_count >= min_sup}, sorted by
   support_count in DESCENDING order // this order = L
3. Create root node of FP-tree, labelled 'null'
4. FOR EACH transaction t in D DO
5.   Select the items of t that are in L
6.   Sort them according to the order of L -> [p | P]
   // p = first (highest-support) item, P = remaining list
7.   CALL Insert_Tree([p | P], root)
8. RETURN FP-tree and header table (linking same-item nodes)
```

```
FUNCTION Insert_Tree([p | P], node)
1. IF node has a child N such that N.item-name = p.item-name THEN
2.   N.count <- N.count + 1
3. ELSE
4.   Create new node N; N.count <- 1;
   N.parent <- node; link N into header table for p
5. IF P is not empty THEN
6.   CALL Insert_Tree(P, N) // recurse on remaining items
```

(b) Mining the FP-tree (pattern growth)

```
ALGORITHM FP_Growth(Tree, alpha, min_sup)
INPUT : Tree (FP-tree), alpha (suffix pattern), min_sup
OUTPUT: complete set of frequent patterns
```

```
1. IF Tree contains a single path P THEN
2.   FOR EACH combination (beta) of nodes in path P DO
3.     generate pattern (beta U alpha)
       with support = min support count in beta
4. ELSE
5.   FOR EACH item ai in the header table of Tree
       (processed bottom-up, i.e. lowest support first) DO
6.     generate pattern beta <- ai U alpha;
       support(beta) <- ai.support
7.     construct beta's conditional pattern base
       (prefix paths of ai in Tree, with counts)
8.     construct beta's conditional FP-tree, Tree_beta
9.     IF Tree_beta is not empty THEN
10.      CALL FP_Growth(Tree_beta, beta, min_sup)
```

2.3 Worked trace on the given dataset (min_sup = 3)

Step 1 — Item support and ordered list L

Item	Support Count	Frequent?
Python	4 (B1,B2,B4,B5)	Yes
Machine Learning	4 (B2,B3,B4,B5)	Yes
Data Science	3 (B1,B3,B4)	Yes

Descending order: L = [Python(4), Machine Learning(4), Data Science(3)] (Python and Machine Learning are tied at 4; Python is listed first by convention/tie-break).

Step 2 — Reorder each transaction by L

Borrow ID	Original Items	Reordered (per L)
B1	Data Science, Python	Python, Data Science
B2	Python, Machine Learning	Python, Machine Learning
B3	Data Science, Machine Learning	Machine Learning, Data Science
B4	Data Science, Python, Machine Learning	Python, Machine Learning, Data Science
B5	Python, Machine Learning	Python, Machine Learning

Step 3 — Insert transactions one at a time

- Insert B1 [Python, Data Science]: root -> Python:1 -> Data Science:1
- Insert B2 [Python, Machine Learning]: Python already a child of root -> Python:2; add child Machine Learning:1 under Python
- Insert B3 [Machine Learning, Data Science]: root has no 'Machine Learning' child yet -> new branch root -> Machine Learning:1 -> Data Science:1
- Insert B4 [Python, Machine Learning, Data Science]: Python:3; its child Machine Learning:2; that node's child Data Science:1
- Insert B5 [Python, Machine Learning]: Python:4; Machine Learning (under Python):3

Step 4 — Final FP-tree structure

```
root
|-- Python:4
|   |-- Data Science:1    (from B1)
|   |-- Machine Learning:3 (from B2,B4,B5)
|       |-- Data Science:1 (from B4)
|-- Machine Learning:1    (from B3)
    |-- Data Science:1    (from B3)
```

Header table:

```
Python    -> node(count 4)
Machine Learning -> node(count 3) -> node(count 1) [total 4]
Data Science -> node(count 1) -> node(count 1) -> node(count 1) [total 3]
```

Step 5 — Mining (bottom-up on header table)

Start with the least-frequent item, Data Science (support 3):

- Conditional pattern base of Data Science: {Python:1}, {Python, Machine Learning:1}, {Machine Learning:1}
- Aggregated: Python:2, Machine Learning:2 — both below min_sup(3), so Data Science's conditional FP-tree is empty; the only frequent pattern emitted here is {Data Science:3} itself.

Next, Machine Learning (support 4):

- Conditional pattern base: {Python:3} (from the Python->Machine Learning path), plus the standalone branch contributes nothing extra to Python's count.
- Python:3 meets min_sup -> conditional FP-tree = single node Python:3 -> frequent pattern {Python, Machine Learning} with support 3.

Finally, Python (support 4) — already frequent as {Python}.

Final frequent patterns (min_sup = 3):

- {Python} : 4
- {Machine Learning} : 4
- {Data Science} : 3
- {Python, Machine Learning} : 3

Note that FP-Growth reached the same conclusion as Apriori would, but without ever constructing or testing an explicit candidate itemset — the FP-tree's shared prefixes (both B2, B4 and B5 share the Python -> Machine Learning path) did all the counting work implicitly.

Question 3 — Association Rule Generation (Frequently Purchased Services)

Frequent itemsets already given (mining phase assumed complete):

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Assume a minimum confidence threshold of min_conf = 70%.

3.1 Concept

Once all frequent itemsets and their support counts are known, rule generation is a second, independent phase: for every frequent itemset l , every non-empty proper subset s of l is treated as a rule antecedent, and the rest ($l - s$) as the consequent. The rule $s \rightarrow (l - s)$ is kept only if its confidence, defined as $\text{support}(l) / \text{support}(s)$, meets min_conf. Because $\text{support}(l) \leq \text{support}(s)$ always, confidence never needs a fresh database scan — it is computed purely from the counts already collected during mining.

3.2 Pseudocode

```

ALGORITHM Generate_Association_Rules(L, min_conf)
INPUT : L      // all frequent itemsets with their support counts
        min_conf // minimum confidence threshold
OUTPUT: R      // set of strong association rules

```

1. $R \leftarrow$ empty set
2. FOR EACH frequent itemset l in L , where $|l| \geq 2$ DO
3. FOR EACH non-empty proper subset s of l DO
4. $\text{confidence} \leftarrow \text{support_count}(l) / \text{support_count}(s)$
5. IF $\text{confidence} \geq \text{min_conf}$ THEN
6. $\text{rule} \leftarrow s \rightarrow (l - s)$

```

7.         rule.support  <- support_count(l)
8.         rule.confidence <- confidence
9.         R <- R U {rule}
10. RETURN R

```

// Efficient variant: generate rules level-wise from consequents of
// size 1 upward, and prune early --
// if a rule with consequent c fails min_conf, every rule whose
// consequent is a SUPERSET of c (from the same l) also fails,
// by the anti-monotonicity of confidence w.r.t. consequent size.

3.3 Worked trace on the given dataset (min_conf = 70%)

Only itemsets of size ≥ 2 can generate rules: {Internet, Streaming} (support 3) and {Streaming, Cloud Storage} (support 2).

From {Internet, Streaming} — support(l) = 3

Candidate Rule	Confidence Calculation	Confidence	Kept? ($\geq 70\%$)
Internet -> Streaming	$3 / \text{support}(\{\text{Internet}\}) = 3/4$	75%	Yes
Streaming -> Internet	$3 / \text{support}(\{\text{Streaming}\}) = 3/4$	75%	Yes

From {Streaming, Cloud Storage} — support(l) = 2

Candidate Rule	Confidence Calculation	Confidence	Kept? ($\geq 70\%$)
Streaming -> Cloud Storage	$2 / \text{support}(\{\text{Streaming}\}) = 2/4$	50%	No
Cloud Storage -> Streaming	$2 / \text{support}(\{\text{Cloud Storage}\}) = 2/3$	66.7%	No

Final strong rules (min_conf = 70%):

- Internet -> Streaming (support 3, confidence 75%)
- Streaming -> Internet (support 3, confidence 75%)

Both rules generated from {Streaming, Cloud Storage} are filtered out — this illustrates why confidence filtering matters even after support-based pruning: an itemset can be frequent overall while the directional implication between its members is still weak.

Question 4 — Constraint-Based Frequent Itemset Mining (Student Course Enrollment)

Dataset given:

Student ID	Courses Enrolled
S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Constraint: only itemsets that contain the item "Python" are of interest. Assume min_sup = 2 (40%).

4.1 Concept

Constraint-based mining pushes a user-specified predicate — here, "itemset must contain Python" — directly inside the mining process instead of filtering results afterward. A constraint of the form "itemset contains item X" is succinct and anti-monotone with respect to that item, so it can be used exactly like the support threshold to prune the search space early: any candidate that does not (and, once formed, never will) contain Python can be discarded immediately, along with all of the joins and subset-checks that would otherwise have been spent on it.

4.2 Pseudocode

```
ALGORITHM Constrained_Apriori(D, min_sup, C)
INPUT : D      // transaction database
        min_sup // minimum support count
        C      // constraint: itemset MUST CONTAIN item 'Python'
OUTPUT: L      // frequent itemsets satisfying C

1. C1 <- {every single item in D}
2. L1 <- {c in C1 | support_count(c) >= min_sup}
3. k <- 2
4. WHILE L(k-1) is not empty DO
5.   Ck <- APRIORI-GEN(L(k-1))
6.   // ---- CONSTRAINT PUSH-DOWN ----
7.   Ck <- {c in Ck | 'Python' is an element of c}
      // discard every candidate that lacks Python BEFORE
      // the database scan -- saves counting effort
8.   FOR EACH transaction t in D DO
9.     IF 'Python' not in t THEN
10.      CONTINUE // t cannot support any
                 // candidate left in Ck; skip scanning it
11.   Ct <- SUBSET(Ck, t)
12.   FOR EACH candidate c in Ct DO
13.     c.count <- c.count + 1
14.   Lk <- {c in Ck | c.count >= min_sup}
15.   k <- k + 1
16. RETURN L <- Union of all Lk // every itemset in L contains Python
```

4.3 Worked trace on the given dataset (min_sup = 2)

Iteration 1 — C1 / L1

Itemset	Support Count	Frequent?
{Python}	4 (S1,S2,S4,S5)	Yes
{Database}	3 (S1,S3,S4)	Yes
{AI}	3 (S2,S3,S4)	Yes

The constraint only removes itemsets that are already invalid to keep, so at $k = 1$ nothing is discarded yet (a single item cannot fail an 'other item present' style test until combined, but note {Python} itself trivially satisfies C).

Iteration 2 — Candidate generation with constraint push-down

Plain Apriori-Gen would produce three candidates: {Python,Database}, {Python,AI}, {Database,AI}. Applying the constraint immediately removes {Database, AI}, because it does

not contain Python — this candidate, and the transactions S3 alone (which contains neither Python), never need to be scanned at all.

Candidate	Contains Python?	Support Count	Frequent? (≥ 2)
{Python, Database}	Yes	2 (S1,S4)	Yes
{Python, AI}	Yes	2 (S2,S4)	Yes
{Database, AI}	No — pruned by constraint before counting	—	—

$L2 = \{ \{ \text{Python, Database} \}, \{ \text{Python, AI} \} \}$.

Iteration 3 — Termination

Joining {Python,Database} and {Python,AI} gives the single candidate {Python, Database, AI}.

Candidate	Support Count	Frequent? (≥ 2)
{Python, Database, AI}	1 (S4 only)	No — pruned

C3 fails support, so the algorithm terminates.

Final constrained frequent itemsets (all containing Python, $\min_sup = 2$):

- {Python} : 4
- {Python, Database} : 2
- {Python, AI} : 2

Effect of the constraint: {Database, AI} (support 3, which would ordinarily be frequent and require further joining) was eliminated purely by the item-membership predicate before any counting was done. In a large catalogue this constraint prunes an entire region of the itemset lattice — every itemset built without Python — cutting both the number of candidates generated and the number of database scans needed.

Question 5 — Multilevel Association Rule Mining (Hospital Treatment Hierarchy)

Dataset given:

Patient ID	Treatments Received
P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Concept hierarchy (given implicitly by the dataset):

Level 0 (root): Healthcare

Level 1 (department): Cardiology Orthopedics

Level 2 (procedure): ECG

(ECG is a child of Cardiology only)

Assume a (uniform) $\min_sup = 2$ and $\min_conf = 60\%$ at every level.

5.1 Concept

Multilevel association rule mining exploits a concept hierarchy over the items so that rules can be discovered not just among raw, specific items but also among the more general categories that summarize them. It typically uses reduced/uniform support thresholds per level and a top-

down, progressive-deepening search: frequent itemsets are found at Level 1 first, and only the children (Level 2 descendants) of the items that turned out to be frequent at Level 1 are examined next. Items whose parent was infrequent are never explored at the lower level, since they cannot contribute a meaningful, well-supported specialization. This exposes rules at multiple granularities — e.g. a strong general rule that does not hold at the specific level, or a specific rule that a flat (single-level) miner would never surface because the specific item alone is too rare.

5.2 Pseudocode

ALGORITHM Multilevel_Apriori(D, Hierarchy, min_sup_list, min_conf)

INPUT : D // transactions, each item taggable to a hierarchy node

Hierarchy // concept tree, root = most general

min_sup_list // support threshold assigned per level (can be uniform

// or reduced/level-specific)

min_conf

OUTPUT: multilevel frequent itemsets and rules

1. level <- 1
2. Encode every transaction with ALL ancestors of its items
(e.g. an ECG entry is logically also Cardiology and Healthcare)
3. F(1) <- Find_Frequent_1_Itemsets(D, level=1, min_sup_list[1])
// run Apriori / FP-Growth restricted to Level-1 concepts
4. WHILE F(level) is not empty AND a deeper level exists DO
5. Candidates(level+1) <- children, in Hierarchy, of ONLY the
 items that appear in F(level)
 // ---- progressive deepening / pruning ----
 // descendants of an infrequent parent are skipped entirely
6. F(level+1) <- Apriori_On(Candidates(level+1), D,
 min_sup_list[level+1])
7. level <- level + 1
8. FOR EACH level L with F(L) not empty DO
9. Generate_Association_Rules(F(L), min_conf)
 // rules may also be generated ACROSS levels, e.g.
 // {parent-item} -> {child-item}, to relate a general
 // category to one of its specific specializations
10. RETURN all frequent itemsets and rules found at every level

5.3 Worked trace on the given dataset

Level 0 — root

{Healthcare} appears in all five records -> support = 5 (100%). Because every patient trivially receives 'Healthcare', this level carries no discriminating information by itself but anchors the hierarchy for cross-level rules.

Level 1 — department

Itemset	Support Count	Frequent? (≥ 2)
{Cardiology}	3 (P1,P3,P4)	Yes
{Orthopedics}	2 (P2,P5)	Yes

F(1) = { {Cardiology}, {Orthopedics} } — both departments qualify, so both are expanded to Level 2.

Level 2 — procedure (progressive deepening)

Only the children of frequent Level-1 nodes are examined. ECG is a child of Cardiology, which was frequent, so ECG is examined; Orthopedics has no Level-2 children listed in this dataset, so nothing is generated under it (no wasted search).

Itemset	Support Count	Frequent? (≥ 2)
{ECG}	2 (P1,P4)	Yes

$F(2) = \{ \{ \text{ECG} \} \}$, and note {ECG} is always accompanied by {Cardiology} in this data, i.e. {Cardiology, ECG} also has support 2.

Cross-level rule generation (min_conf = 60%)

Rule	Confidence Calculation	Confidence	Kept?
Healthcare -> Cardiology	3 / 5	60%	Yes (borderline)
Cardiology -> ECG	2 / 3	66.7%	Yes
Healthcare -> Orthopedics	2 / 5	40%	No
Healthcare -> ECG	2 / 5	40%	No

Interpretation:

- The rule Cardiology -> ECG (66.7% confidence) is a specific, actionable finding that a flat, single-level miner over raw treatment codes could easily miss if ECG alone were too infrequent to reach a global support threshold.
- The rule Healthcare -> Orthopedics fails confidence, showing that a very general antecedent (present in 100% of records) does not guarantee strong rules toward every one of its descendants — Orthopedics needs its own direct evidence.
- Progressive deepening meant Orthopedics's (non-existent) children were never even considered, saving a wasted database pass, exactly as the constraint push-down did in Question 4.

Summary — How the Five Techniques Relate

Q	Technique	Core Idea	Key Pruning / Efficiency Mechanism
Q1	Apriori	Level-wise candidate generate-and-test	Downward-closure (anti-monotone) property
Q2	FP-Growth	Compress DB into FP-tree; mine by pattern growth	No candidate generation; shared-prefix compression
Q3	Rule Generation	Derive rules from already-frequent itemsets	Confidence is anti-monotone w.r.t. consequent size
Q4	Constraint-Based Mining	Push a user predicate into the mining loop	Item-membership constraint prunes non-matching candidates early
Q5	Multilevel Mining	Mine frequent patterns across a concept hierarchy	Progressive deepening: only expand children of frequent parents